Lecture Notes: TwinCAT 3

Standard Functions and Function Blocks (Tc2_Standard)

March 17, 2026

Contents

1 Introduction to the IEC 61131-3 Standard **2**

2 Timer Function Blocks **2**

2.1 Timer On Delay (TON) 2

2.2 Timer Off Delay (TOF) 4

2.3 Use Case: Motor Safety 5

3 Edge Detection Function Blocks **6**

3.1 Rising Edge (R_TRIG) 6

3.2 Falling Edge (F_TRIG) 7

3.3 Use Case: Preventing Message Flooding 9

4 String Manipulation: CONCAT **10**

5 Library Management **12**

Quick Reference Card **12**

1 Introduction to the IEC 61131-3 Standard

This lecture focuses on the basic functions and function blocks defined by the **IEC 61131-3 standard**, which are essential for every PLC programmer. In TwinCAT 3, these are found in the **Tc2_Standard library**. Because these functions are standardized, code written using them is **highly portable** across different PLC environments.

Category	Function Blocks	Purpose
Timers	TON, TOF, TP	Time-based control
Counters	CTU, CTD, CTUD	Event counting
Edge Detection	R_TRIG, F_TRIG	Signal transition detection
String Operations	CONCAT, LEFT, RIGHT, MID	Text manipulation

Table 1: Key IEC 61131-3 Standard Library Components

> **Note:** The Tc2_Standard library is automatically referenced in most new TwinCAT projects. If you encounter "undefined symbol" errors for timers or triggers, verify that the library reference exists under References in your PLC project.

2 Timer Function Blocks

Timers are used to manage time-sensitive operations, such as waiting for a process to complete or limiting the duration of an action.

2.1 Timer On Delay (TON)

- **Purpose:** The `TON` function block turns its output (`Q`) on only after a specified time delay (`PT`).
- **Inputs:**
  - **IN:** A rising edge (transition from FALSE to TRUE) starts the timer. A falling edge resets the timer.
  - **PT (Preset Time):** The duration of the delay (e.g., `T#3s`).
- **Outputs:**
  - **Q:** Becomes TRUE once the elapsed time reaches the preset time.
  - **ET (Elapsed Time):** Shows how much time has passed since the timer started.
- **Behavior:** If the input `IN` goes low before the preset time is reached, the timer resets and `Q` never goes high.

Example: TON Declaration and Call

```
// PROGRAM: MAIN
// Demonstrates TON (Timer On Delay) function block usage
// The timer output Q goes TRUE after the preset time elapses

PROGRAM MAIN
VAR
```

```
    // ====================================================
    // TON Instance Declaration
    // The timer can be initialized with a preset time (PT)
    // This sets the default delay duration
    // ====================================================
    fbTimer      : TON := (PT := T#10S);  // 10 second delay timer

    // Alternative declaration without initialization
    fbTimer2     : TON;                    // PT must be set when called

    // Additional timers for different purposes
    fbStartupDelay  : TON := (PT := T#5S);   // 5 second startup delay
    fbTimeoutTimer  : TON := (PT := T#30S);  // 30 second timeout

    // Control variables
    bStartProcess    : BOOL;      // Input: Start button
    bProcessActive   : BOOL;      // Output: Process is running
    bTimerComplete   : BOOL;      // Output: Timer has finished

    // Monitoring variables
    tElapsedTime     : TIME;      // Current elapsed time
END_VAR

// ====================================================
// Basic TON Call
// The timer starts counting when IN transitions to TRUE
// Q becomes TRUE after PT time has elapsed
// ====================================================

// Call the timer with the start button as trigger
fbTimer(IN := bStartProcess);

// Read the outputs
bTimerComplete := fbTimer.Q;       // TRUE when 10 seconds elapsed
tElapsedTime := fbTimer.ET;        // Current elapsed time (0 to PT)

// ====================================================
// Alternative: Set PT dynamically when calling
// ====================================================
fbTimer2(
    IN := bStartProcess,
    PT := T#15S                    // Override preset time
);

// ====================================================
// Practical Example: Startup Delay
// Wait 5 seconds after start button before activating
// ====================================================
fbStartupDelay(IN := bStartProcess);

IF fbStartupDelay.Q THEN
    bProcessActive := TRUE;        // Activate after delay
END_IF

// ====================================================
// Using ET for progress indication
```

```
// ET counts from T#0S up to PT while IN is TRUE
// ==================================================
// tElapsedTime can be used for:
// - Progress bars on HMI
// - Logging how long a process takes
// - Conditional logic based on partial timing
```

```
// Extended example: Multiple timers working together

VAR
    fbWarmupTimer    : TON := (PT := T#2M);     // 2 minute warmup
    fbCycleTimer     : TON := (PT := T#500MS); // 500ms cycle
    fbAlarmDelay     : TON := (PT := T#5S);     // 5 second alarm delay

    bMachineOn       : BOOL;
    bOverTemp        : BOOL;
    bWarmupComplete  : BOOL;
    bAlarmActive     : BOOL;
END_VAR

// Warmup timer - machine needs 2 minutes before operation
fbWarmupTimer(IN := bMachineOn);
bWarmupComplete := fbWarmupTimer.Q;

// Alarm delay - only trigger alarm if overtemp persists for 5
    seconds
// This prevents false alarms from brief temperature spikes
fbAlarmDelay(IN := bOverTemp AND bWarmupComplete);
bAlarmActive := fbAlarmDelay.Q;
```

Time	0s	1s	2s	3s	4s	5s	6s	7s
IN	0	1	1	1	1	1	0	0
ET	0	1s	2s	3s	3s	3s	0	0
Q (PT=3s)	0	0	0	1	1	1	0	0

Figure 1: TON Timing Diagram (PT = 3 seconds)

2.2 Timer Off Delay (TOF)

- **Purpose:** The inverse of `TON`. It counts time when the input `IN` is **low**.
- **Trigger:** It is triggered by a **falling edge**.
- **Output Q:** Goes TRUE immediately when `IN` is high and only drops to FALSE after the input has been low for the duration of `PT`.

Timer	**Starts When**	**Q Goes HIGH**	**Q Goes LOW**
TON	IN rises	After PT elapsed	When IN falls
TOF	IN falls	When IN rises	After PT elapsed
TP	IN rises	When IN rises	After PT elapsed

Table 2: Comparison of Timer Function Blocks

2.3 Use Case: Motor Safety

Timers can be used as a **safety feature** to prevent equipment from running indefinitely due to system errors. For instance, if an HMI button controlling a motor gets "stuck" in a high state (due to a crash or communication error), a `TON` can ensure the motor moves for a maximum of 5 or 10 seconds before automatically stopping.

Example: Motor Safety Logic

```
// PROGRAM: MotorSafetyControl
// Demonstrates using a timer as a safety limit for motor operation
// Prevents motor from running indefinitely if HMI crashes

PROGRAM MotorSafetyControl
VAR
    // ====================================================
    // Safety Timer
    // Limits maximum motor run time to prevent damage
    // ====================================================
    fbSafetyTimer    : TON := (PT := T#10S);  // 10 second maximum run time

    // Inputs
    bOperatorClick   : BOOL;      // HMI button: Operator commands motor to move
    bLimitSwitch     : BOOL;      // Physical limit switch (motor reached position)
    bSafetyOK        : BOOL := TRUE;  // Safety circuit status

    // Outputs
    bMoveMotor       : BOOL;      // Output to motor contactor/drive
    bTimeoutAlarm    : BOOL;      // Alarm: Motor ran too long

    // Status
    bMotorMoving     : BOOL;      // Motor is currently moving
END_VAR

// ====================================================
// Start the safety timer when operator clicks the button
// The timer runs as long as the operator holds the button
// ====================================================
fbSafetyTimer(IN := bOperatorClick);

// ====================================================
// MOTOR SAFETY LOGIC
// Motor runs when:
//   - Operator is pressing the button (bOperatorClick = TRUE)
//   - AND safety timer has NOT elapsed (fbSafetyTimer.Q = FALSE)
//   - AND safety circuit is OK
//   - AND limit switch not reached
// ====================================================
bMoveMotor := bOperatorClick
              AND NOT fbSafetyTimer.Q
              AND bSafetyOK
              AND NOT bLimitSwitch;

// Alternative simplified version:
```

```
// bMoveMotor := (bOperatorClick AND NOT fbSafetyTimer.Q);

// ====================================================
// Generate alarm if timer elapsed while button still pressed
// This indicates the HMI may be stuck or motor is jammed
// ====================================================
bTimeoutAlarm := fbSafetyTimer.Q AND bOperatorClick;

// Status output
bMotorMoving := bMoveMotor;

// ====================================================
// BEHAVIOR EXPLANATION:
// 1. Operator presses button -> Motor starts, timer starts
// 2. If operator releases before 10s -> Motor stops, timer resets
// 3. If 10 seconds elapse with button held -> Motor stops automatically
// 4. Operator must release and re-press button to restart
// ====================================================
```

```
// Extended example with multiple safety features

VAR
    fbMaxRunTimer    : TON := (PT := T#30S);    // Maximum run time
    fbJogTimer       : TON := (PT := T#2S);     // Jog mode limit

    bJogForward      : BOOL;
    bJogReverse      : BOOL;
    bAutoMode        : BOOL;
    bMotorFwd        : BOOL;
    bMotorRev        : BOOL;
END_VAR

// Jog mode - limited to 2 seconds per press
fbJogTimer(IN := bJogForward OR bJogReverse);

// Motor outputs with safety limits
bMotorFwd := bJogForward AND NOT fbJogTimer.Q AND NOT bJogReverse;
bMotorRev := bJogReverse AND NOT fbJogTimer.Q AND NOT bJogForward;
```

Warning: Always implement safety timers for manual motor control. If an HMI freezes or loses communication while a button is pressed, the motor could run indefinitely, causing equipment damage, product loss, or personnel injury.

3 Edge Detection Function Blocks

Edge detectors are used to identify the exact moment a signal changes state.

3.1 Rising Edge (R_TRIG)

- **Purpose:** Detects a transition from FALSE to TRUE.
- **Behavior:** The output `Q` stays TRUE for only **one PLC cycle**. To re-trigger it, the input must return to FALSE and then go TRUE again.

- **Common Use:** Detecting an operator button press or raising an alarm only once when a condition is met.

3.2 Falling Edge (F_TRIG)

- **Purpose:** Detects a transition from TRUE to FALSE.
- **Behavior:** Like `R_TRIG`, the output `Q` is high for exactly one PLC cycle upon detecting the falling edge.

Example: Edge Trigger Counter

```
// PROGRAM: EdgeDetectionDemo
// Demonstrates R_TRIG and F_TRIG for edge detection
// Essential for counting events and preventing repeated actions

PROGRAM EdgeDetectionDemo
VAR
    // ====================================================
    // Edge Detection Function Block Instances
    // Each instance maintains its own state (previous CLK value)
    // ====================================================
    rtButtonPress    : R_TRIG;     // Rising edge detector for button
    ftButtonRelease  : F_TRIG;     // Falling edge detector
    rtSensorTrigger  : R_TRIG;     // Rising edge for sensor

    // Inputs
    bButton          : BOOL;       // Pushbutton input
    bSensorSignal    : BOOL;       // Sensor input (e.g., part detected)

    // Counters
    nTriggerCounter  : INT := 0;   // Counts rising edges (button presses)
    nReleaseCounter  : INT := 0;   // Counts falling edges (button releases)
    nPartCount       : INT := 0;   // Counts parts detected by sensor

    // Status flags
    bJustPressed     : BOOL;       // TRUE for one cycle when pressed
    bJustReleased    : BOOL;       // TRUE for one cycle when released
END_VAR

// ====================================================
// Call the rising edge detector
// CLK is the signal to monitor for edges
// ====================================================
rtButtonPress(CLK := bButton);

// ====================================================
// Use the Q output to increment a counter
// Q is TRUE for exactly ONE PLC cycle on each rising edge
// This ensures the counter increments once per button press
// ====================================================
IF rtButtonPress.Q THEN
    // This block executes exactly ONCE per button press
    nTriggerCounter := nTriggerCounter + 1;
```

```
    bJustPressed := TRUE;
ELSE
    bJustPressed := FALSE;
END_IF

// ====================================================
// Falling edge detection - detect button release
// ====================================================
ftButtonRelease(CLK := bButton);

IF ftButtonRelease.Q THEN
    // This block executes exactly ONCE per button release
    nReleaseCounter := nReleaseCounter + 1;
    bJustReleased := TRUE;
ELSE
    bJustReleased := FALSE;
END_IF

// ====================================================
// Practical Example: Part Counter
// Count parts as they pass a sensor
// ====================================================
rtSensorTrigger(CLK := bSensorSignal);

IF rtSensorTrigger.Q THEN
    nPartCount := nPartCount + 1;
    // Could also log the event, update HMI, etc.
END_IF
```

```
// Extended example: Multiple edge detectors for different purposes

VAR
    rtStartButton   : R_TRIG;
    rtStopButton    : R_TRIG;
    rtResetButton   : R_TRIG;
    ftSafetyGate    : F_TRIG;    // Detect when gate opens (signal
    drops)

    bStart          : BOOL;
    bStop           : BOOL;
    bReset          : BOOL;
    bGateClosed     : BOOL;

    bMachineRunning : BOOL;
    nCycleCount     : UDINT := 0;
    bGateAlarm      : BOOL;
END_VAR

// Start/Stop logic using edge detection
rtStartButton(CLK := bStart);
rtStopButton(CLK := bStop);
rtResetButton(CLK := bReset);
ftSafetyGate(CLK := bGateClosed);

// Start machine on start button press
IF rtStartButton.Q AND bGateClosed THEN
    bMachineRunning := TRUE;
END_IF
```

```
// Stop machine on stop button press
IF rtStopButton.Q THEN
    bMachineRunning := FALSE;
END_IF

// Alarm if gate opens while machine is running
IF ftSafetyGate.Q AND bMachineRunning THEN
    bGateAlarm := TRUE;
    bMachineRunning := FALSE;  // Emergency stop
END_IF

// Reset alarm
IF rtResetButton.Q THEN
    bGateAlarm := FALSE;
    nCycleCount := 0;
END_IF
```

Cycle	1	2	3	4	5	6	7	8	9
CLK	0	1	1	1	0	0	1	1	0
R_TRIG.Q	0	**1**	0	0	0	0	**1**	0	0
F_TRIG.Q	0	0	0	0	**1**	0	0	0	**1**

Figure 2: Edge Detection Timing Diagram

3.3 Use Case: Preventing Message Flooding

When logging events (e.g., using `ADSLOGSTR`), calling the function directly with a boolean input can result in hundreds of messages per second because the PLC runs cyclically. Using an `R_TRIG` ensures that only **one message is generated** per event (like a button click or temperature threshold), and the signal must be reset before another message can be sent.

```
// Example: Using R_TRIG to prevent log message flooding

VAR
    rtLogTrigger     : R_TRIG;
    bAlarmCondition : BOOL;      // TRUE when temperature too high
END_VAR

rtLogTrigger(CLK := bAlarmCondition);

IF rtLogTrigger.Q THEN
    // This logs ONCE when alarm condition becomes true
    // Not every cycle while it remains true
    ADSLOGSTR(ADSLOG_MSGTYPE_WARN, 'Temperature alarm activated', '')
    ;
END_IF
```

Tip: Always use R_TRIG or F_TRIG when you need to perform an action "once per event" rather than "continuously while condition is true." This is especially important for logging, counting, and state machine transitions.

4 String Manipulation: CONCAT

The `CONCAT` function is used to join two strings together into one.

- **Limitations:** The standard `CONCAT` function works with strings up to **255 characters**.
- **Arbitrary Length:** For strings longer than 255 characters, the `CONCAT2` function should be used instead.

Example: String Concatenation

```
// PROGRAM: StringOperationsDemo
// Demonstrates CONCAT and other string manipulation functions
// Useful for building messages, file paths, and formatted output

PROGRAM StringOperationsDemo
VAR
    // ====================================================
    // String Variables for Concatenation
    // STRING default length is 80 characters
    // STRING(n) specifies maximum length of n characters
    // ====================================================
    s1          : STRING := 'Hello ';        // First string
    s2          : STRING := 'World!';        // Second string
    sResult     : STRING(255);               // Result string (larger buffer)

    // Additional strings for examples
    sPrefix     : STRING := 'Error: ';
    sMessage    : STRING := 'Sensor disconnected';
    sFullMessage: STRING(255);

    // For building file paths
    sBasePath   : STRING := 'C:\Data\';
    sFileName   : STRING := 'log_';
    sExtension  : STRING := '.txt';
    sFullPath   : STRING(255);

    // For dynamic messages
    sProductID  : STRING := 'WIDGET-001';
    sStatus     : STRING := ' - PASSED';
    sLogEntry   : STRING(255);

    // Numeric values to include in strings
    nErrorCode  : INT := 42;
    sErrorCode  : STRING;
END_VAR

// ====================================================
// Basic CONCAT Usage
// Joins two strings into one
// Syntax: result := CONCAT(string1, string2);
// ====================================================
sResult := CONCAT(s1, s2);
// sResult now contains: 'Hello World!'

// ====================================================
```

```
// Building an error message
// ====================================================
sFullMessage := CONCAT(sPrefix, sMessage);
// sFullMessage now contains: 'Error: Sensor disconnected'

// ====================================================
// Multiple concatenations (chained)
// CONCAT only takes 2 arguments, so chain calls for more
// ====================================================
sFullPath := CONCAT(sBasePath, sFileName);
sFullPath := CONCAT(sFullPath, sExtension);
// sFullPath now contains: 'C:\Data\log_.txt'

// Alternative: Nested CONCAT calls
sFullPath := CONCAT(CONCAT(sBasePath, sFileName), sExtension);

// ====================================================
// Building a log entry
// ====================================================
sLogEntry := CONCAT(sProductID, sStatus);
// sLogEntry now contains: 'WIDGET-001 - PASSED'

// ====================================================
// IMPORTANT: Converting numbers to strings
// Use INT_TO_STRING, REAL_TO_STRING, etc.
// ====================================================
sErrorCode := INT_TO_STRING(nErrorCode);
sFullMessage := CONCAT('Error code: ', sErrorCode);
// sFullMessage now contains: 'Error code: 42'
```

```
// Extended example: Building formatted messages

VAR
    sTimestamp      : STRING := '2024-01-15 10:30:00';
    sMachineName    : STRING := 'Conveyor_01';
    sEventType      : STRING := 'FAULT';
    sDescription    : STRING := 'Motor overtemperature';

    sLogLine        : STRING(255);
    sTemp           : STRING(255);
END_VAR

// Build a structured log message:
// "[2024-01-15 10:30:00] Conveyor_01 - FAULT: Motor overtemperature"

sLogLine := CONCAT('[', sTimestamp);
sLogLine := CONCAT(sLogLine, '] ');
sLogLine := CONCAT(sLogLine, sMachineName);
sLogLine := CONCAT(sLogLine, ' - ');
sLogLine := CONCAT(sLogLine, sEventType);
sLogLine := CONCAT(sLogLine, ': ');
sLogLine := CONCAT(sLogLine, sDescription);

// Result: '[2024-01-15 10:30:00] Conveyor_01 - FAULT: Motor
    overtemperature'
```

Function	Syntax	Description
CONCAT	CONCAT(s1, s2)	Joins two strings
LEFT	LEFT(str, n)	Returns first n characters
RIGHT	RIGHT(str, n)	Returns last n characters
MID	MID(str, pos, n)	Returns n chars starting at pos
LEN	LEN(str)	Returns string length
FIND	FIND(str1, str2)	Finds str2 in str1, returns position

Table 3: Common String Manipulation Functions

Note: When concatenating strings, always ensure your result variable has sufficient capacity. If the combined string exceeds the variable's declared length, it will be truncated without warning.

5 Library Management

To access these functions, your TwinCAT project must have a reference to the **Tc2_Standard library**. Most new TwinCAT projects include this reference automatically, but it is necessary for functions like timers and triggers to be recognized by the compiler.

Library	Contents	Required For
Tc2_Standard	TON, TOF, R_TRIG, etc.	Basic PLC functions
Tc2_System	ADSLOGSTR, system functions	Logging, diagnostics
Tc2_Utilities	File I/O, advanced functions	Extended functionality

Table 4: Common TwinCAT Libraries

Tip: To add a library reference, right-click on "References" in your PLC project, select "Add library...", and search for the library name. The Tc2_Standard library should always be present for basic PLC functionality.

Quick Reference Card

Timer Function Blocks

```
// TON - On Delay Timer
fbTON : TON := (PT := T#5S);
fbTON(IN := bTrigger);
// Q goes TRUE after 5 seconds if IN stays TRUE

// TOF - Off Delay Timer
fbTOF : TOF := (PT := T#5S);
fbTOF(IN := bTrigger);
// Q stays TRUE for 5 seconds after IN goes FALSE

// TP - Pulse Timer
fbTP : TP := (PT := T#5S);
fbTP(IN := bTrigger);
```

```
// Q goes TRUE for exactly 5 seconds on rising edge
```

Edge Detection

```
// Rising Edge
rtTrigger : R_TRIG;
rtTrigger(CLK := bInput);
IF rtTrigger.Q THEN
    // Executes once on FALSE->TRUE transition
END_IF

// Falling Edge
ftTrigger : F_TRIG;
ftTrigger(CLK := bInput);
IF ftTrigger.Q THEN
    // Executes once on TRUE->FALSE transition
END_IF
```

String Functions

```
// Concatenation
sResult := CONCAT(s1, s2);

// Substring
sLeft := LEFT(sSource, 5);      // First 5 chars
sRight := RIGHT(sSource, 3);    // Last 3 chars
sMid := MID(sSource, 2, 4);     // 4 chars from pos 2

// Length
nLen := LEN(sSource);
```

Time Literals

Literal	Value
T#100MS	100 milliseconds
T#5S	5 seconds
T#2M30S	2 minutes 30 seconds
T#1H	1 hour
T#1D	1 day

Last Updated: March 17, 2026